

Carlos Antonio Frias Juárez

Parte 1: Métodos HTTP y Gestión de Recursos

Relaciona cada acción de administración de red con el método HTTP más apropiado.

Escribe el método (GET, POST, PUT, PATCH, DELETE) en la columna correspondiente.

Acción sobre el Recurso de Red	Método HTTP
Obtener la lista de todas las VLANs configuradas en un switch.	GET
Crear un nuevo objeto de red (Network Object) en un firewall.	POST
Reemplazar la configuración completa de una interfaz OSPF.	PUT
Eliminar una ruta estática que ya no es necesaria.	DELETE
Modificar únicamente la descripción de una interfaz (cambio parcial).	PATCH

Parte 2: Análisis de Códigos de Estado

Como ingeniero de redes, estás auditando logs de una API RESTCONF. Predice qué código de estado devolvería el controlador en las siguientes situaciones.

1. Un script intenta consultar el estado de la interfaz GigabitEthernet99 , pero dicha interfaz no existe físicamente ni lógicamente en el router. ¿Qué código devuelve la API?

A) 200 OK

B) 404 Not Found

C) 500 Internal Server Error

D) 201 Created

2. Se envía una petición POST para configurar un nuevo túnel VPN, pero el token de autenticación en el encabezado Authorization ha expirado o es inválido.

A) 401 Unauthorized

B) 204 No Content

C) 400 Bad Request

Parte 3: Diagnóstico y Justificación

3. Un administrador de red intenta actualizar la IP de un servidor mediante un método PUT. La API responde con un código 400 Bad Request. Explica dos posibles razones técnicas relacionadas con la estructura de la petición (payload) que podrían causar este error específico:

El error 400 Bad Request significa que el servidor recibió la petición, pero algo en los datos que se enviaron el payload estaba mal. Dos posibles razones:

Razón 1: El JSON tiene errores de escritura.

falta una coma, unas comillas o se dejó una llave sin cerrar. El servidor no puede leerlo y lo rechaza. Razón 2: Los datos no son válidos para la API.

El JSON está bien escrito, pero la IP tiene un formato incorrecto como 192.168.1.300 o falta un campo obligatorio. El servidor entiende el mensaje, pero no puede aceptar esos datos. En resumen: el servidor dice "te entendí, pero lo que me mandaste está mal". Hay que corregir el payload y volver a enviarlo.

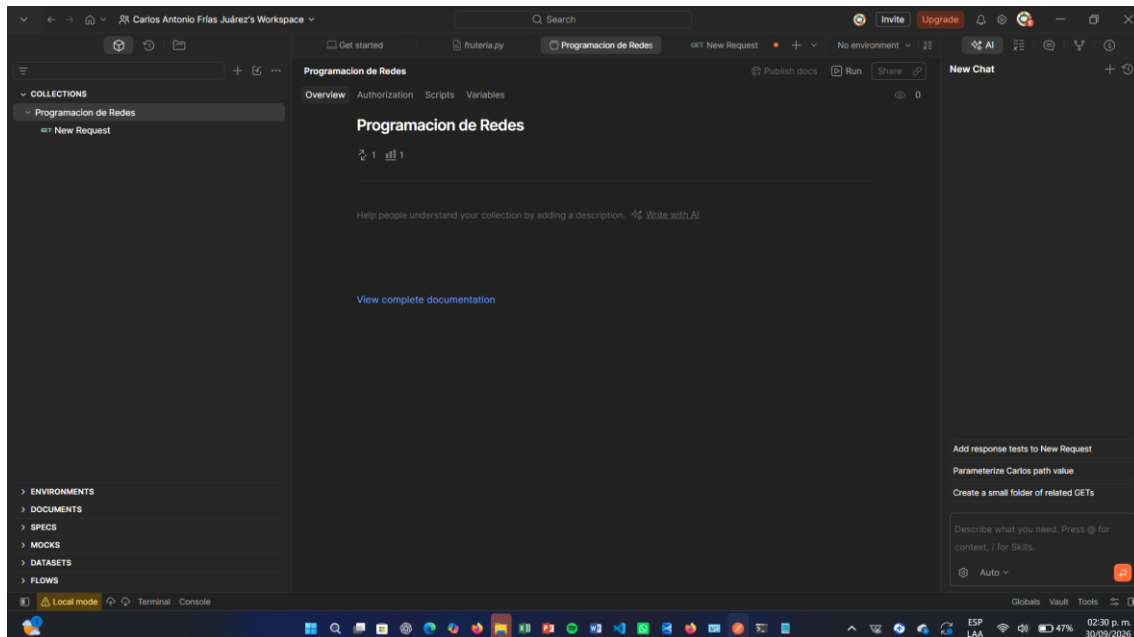
4. ¿Cuál es la diferencia principal entre el uso de POST y PUT al configurar una interfaz de red, considerando la propiedad de idempotencia?

Idempotencia significa que, si repites la misma petición varias veces, el resultado es siempre el mismo que si la hubieras enviado una sola vez.

PUT: reemplaza la configuración de una interfaz específica. Lo envíes una vez o diez, la interfaz queda igual. No se crean duplicados.

POST: crea un recurso nuevo. Si repites la petición, puede crear otro recurso igual o dar error.

En conclusión: PUT se puede repetir sin problema; POST cambia algo nuevo cada vez.



1. Escribe el pseudocódigo o completa el script de Python necesario para procesar la variable inventario_json (que contiene el texto anterior). El script debe filtrar e imprimir únicamente los hostnames de los dispositivos cuyo estado sea 'down'

'''

Mi primer servidor web con Flask y Python

Autor: Carlos Antonio Frias

Fecha: 30 septiembre

'''

```
from flask import Flask
```

```
app = Flask(__name__)
```

```
inventario = [  
    {"hostname": "Core-SW01", "ip": "10.0.0.1", "status": "up"},  
    {"hostname": "Dist-SW02", "ip": "10.0.0.2", "status": "down"},  
    {"hostname": "Access-SW03", "ip": "10.0.0.3", "status": "up"},  
    {"hostname": "Edge-R01", "ip": "172.16.1.1", "status": "down"}  
]
```

```
@app.route("/")
```

```
def hello_world():  
    return "<p>Hello, World!</p>"
```

```
@app.route('/<nombre>')
```

```
def saludo(nombre):  
    return f"<h1>Hola {nombre}</h1>"
```

```
@app.route('/dispositivo/<device>')
```

```
def buscar_dispositivo(device):  
    for d in inventario:  
        if d["hostname"].lower() == device.lower():  
            return f"Nombre: {d['hostname']} | IP: {d['ip']} | Status: {d['status']}"  
    return "Dispositivo no encontrado", 404
```

```
if __name__ == "__main__":  
    app.run(debug=True, port=5050)
```